

CURSO DE CAPACITAÇÃO

POSTGRESQL

Banco de Dados Relacional Open Source

Módulo Básico ao Intermediário

Instrutor: José Charles Curvina de Menezes
Auxiliar Tecnológico / Técnico de Informática

Carga Horária: 40 horas

Local: Laboratório de Informática / J2CM

Data: Agosto de 2025

ROTEIRO DO CURSO DE BD POSTGRESQL - J2CM

```
-- ++++++
47. [PostgreSQL] Instalação do PostgreSQL
1. Introdução
2. Instalação do Postgresql
3. Criando as tabelas do BD Northwind
-- ++++++
```

1° - CRIE UM REPOSITÓRIO NO GITHUB.COM

2° - COPIAR A URL PARA CRIAÇÃO DO REPOSITÓRIO LOCAL

- CODIGO

- Copiar a URL para criação do Repositório LOCAL

- <https://github.com/JCharlesCM-hub/Cursos-SQL.git>

3° - ABRIR A PASTA DO REPOSITÓRIO LOCAL COM O VISUAL CODE - VSCODE

- Vá até a pasta criada para o repositório local com o Windows Explorer

- Selecione a barra de endereço e digite \$ CMD

- Depois digite \$ CODE . (abrirar o VSCODE)

4° - AO ABRIR O VSCODE

- Clicar em Terminal / Novo Terminal

- Ao abrir o terminal no lado direito na seta(+V) para abrir o Terminal bash

- Clique em bash e abrirá o terminal \$

- \$ git clone <https://github.com/jcharlescm/Curso-SQL.git>

4° - INSTALAR O POSTGRESQL

- No GOOGLE pesquisar por POSTGRESQL

- <https://www.postgresql.org/download/>

- Downloads da Comunidade POSTGRESQL para Windows

- Baixar Arquivo ZIP EDB

- "Baixe o instalador" certificado pela EDB para todas as versões suportadas do Postgres.

- Binários do instalador versão 17.5 para

windows

- Copiei para: F:

\MeusArq\Cursos\Curso_SQL\Banco-POSTGRESQL

- Instalador POSTGRESQL para Windows

- Windows (x86, 32 bits), instalador EDB

- Não fazer login:

- Não, obrigado, apenas inicie meu download.

5° - Baixa e copiar para:

F:\MeusArq\Cursos\Curso_SQL\Banco-POSTGRESQL

- Tutorial de POSTGRESQL

- <https://www.devmedia.com.br/postgresql-tutorial/>

- 6° - Executar o arquivo e next, next ...
- Instalar a versão completa
Senha: j2cm1928
- Next, next... informar a senha novamente e checar, next..
- 7° - Abrir o POSTGRESQL ou pgAdmin
- Clicar em pesquisa e digitar pgAdmin
- Ao abrir o pgAdmin clicar em Servers(parte superior esquerda > seta) e informar a senha: j2cm1928
- 8° - Criar Object ou Database
- Clicar em Crie / Database
- Colocar o nome de "northwind" e SAVE, criou o BD northwind.
- https://github.com/pthom/northwind_psql -- Link para a importação das tabelas
- Clica no arquivo northwind.sql no github para abrir e copiar os codigos para o POSTGRESQL
- Clicar em RAW, na parte superior direita
- Selecione o texto do arquivo northwind.sql e copie
- No pgAdmin clicar no banco e depois em "Query Tool" e colar o conteudo
- Executar o script ou F5
- Clicar com o btn direito no BD northwind e na opção REFRESH, para atualizar
- Clica com btn esquerdo no BD northwind / SCHEMAS / TABLES, estam as tabelas importtadas - ok
- Com o btn direito pode ver o conteudo das tabelas em "View / Edit Data / Rows "

-- ++++++

48. [PostgreSQL] Criando queries básicas no Postgres

1. SELECT FROM - Selecionando dados das tabelas

2. Comentários no PostgreSQL

3. SELECT AS - Aliasing (renomeando) colunas

4. SELECT LIMIT - Limitando a quantidade de linhas de uma query

5. SELECT DISTINCT - Selecionando valores distintos de uma coluna
Feedback do módulo.

-- ++++++

/* COMENTÁRIOS */

-- Para comentários de linha única, usa-se "--" no início da linha.

-- Para comentários de múltiplas linhas, utiliza-se "/*" para iniciar e "*/" para finalizar.

/*

Selecionar dados das tabelas.

*/

SELECT * FROM categories;

SELECT * FROM customers;

SELECT * FROM employees;

```
SELECT category_id, category_name
FROM categories;
```

```
SELECT first_name, last_name
FROM employees;
```

```
SELECT product_id, product_name, unit_price
FROM products;
```

```
-- 3. SELECT AS - Aliasing (renomeando) colunas e tabela
SELECT
```

```
    pr.product_id AS id_produto,
    pr.product_name AS nome_produto,
    pr.unit_price AS preco_unitario
FROM products AS pr;
```

```
SELECT * FROM orders AS ordem;
SELECT ordem.* FROM orders AS ordem;
```

```
-- 4. SELECT LIMIT - Limitando a quantidade de linhas de uma query
SELECT * FROM orders LIMIT 5;
SELECT * FROM categories LIMIT 10;
```

```
-- 5. SELECT DISTINCT - Selecionando valores distintos de uma coluna
-- Consultar as profissões(contact_title) distintas da tabela
customers:
```

```
SELECT * FROM customers;
SELECT DISTINCT contact_title FROM customers;
SELECT DISTINCT(contact_title) FROM customers;
```

```
/* MÓDULO 49 - POSTGRESQL
```

```
49. [PostgreSQL] Filtros no Postgres
```

```
1. WHERE - Filtros com textos
```

```
2. WHERE - Filtros com números e datas
```

```
3. WHERE - Operadores AND e OR
```

```
4. WHERE e LIKE - Filtros especiais com textos
```

```
5. WHERE e IN - Uma alternativa aos múltiplos ORs
```

```
6. WHERE e BETWEEN - Filtrando intervalos
```

```
*/
```

```
-- ## WHERE ## --
```

```
-- Permite criar filtros nas consultas. Obs.: O Postgres é case-
sensitive.
```

```
-- 1. Selecione a tabela customers
```

```
-- a) Crie um filtro para que sejam mostrados apenas os clientes com
contact_title = 'Owner'.
```

```
SELECT * FROM customers
WHERE contact_title = 'Owner';
```

```
-- b) Crie um filtro para que sejam mostrados apenas os clientes do
país France.
```

```
SELECT * FROM customers
WHERE country = 'France';
```

-- 2. Selecione a tabela products

-- a) Crie um filtro para que sejam mostrados os produtos com estoque igual a zero (units_in_stock).

-- b) Crie um filtro para mostrar os produtos com unit_price maior ou igual a 50.

```
SELECT * FROM products
WHERE units_in_stock = 0;
```

```
SELECT * FROM products
WHERE unit_price ≥ 50
ORDER BY unit_price DESC;
```

-- 3. Selecione a tabela orders

-- Crie um filtro para mostrar apenas os pedidos feitos depois do dia '01/01/1998'.

```
SELECT * FROM orders
WHERE order_date ≥ '1998-01-01';
```

-- ## WHERE + AND e OR ## --

-- Permite criar filtros com mais de uma coluna.

-- AND: o filtro será realizado se todas as condições forem verdadeiras.

-- OR: o filtro será realizado desde que PELO MENOS 1 condição seja verdadeira.

-- 1. Selecione a tabela customers

-- a) Crie um filtro para que sejam mostrados apenas os clientes com contact_title = 'Owner' E do país France.

```
SELECT * FROM customers
WHERE contact_title = 'Owner' AND country = 'France';
```

-- b) Crie um filtro para que sejam mostrados apenas os clientes do Mexico OU France.

```
SELECT * FROM customers
WHERE country = 'Mexico' OR country = 'France';
```

-- ## WHERE + LIKE ## --

-- Permite criar filtros especiais de textos.

-- 1. Selecione a tabela products

-- a) Quais são os produtos medidos em boxes?

```
SELECT * FROM products
WHERE quantity_per_unit LIKE '%boxes%';
```

-- b) Quais são os produtos medidos em ml?

```
SELECT * FROM products
WHERE quantity_per_unit LIKE '%ml%';
```

```
-- ## WHERE + IN ## --
-- Essa combinação é uma alternativa ao uso de múltiplos OR.

-- 1. Selecione a tabela customers
-- a) Quais clientes são dos países 'Mexico', 'UK', 'Canada'
(utilize o método OR para resolver)
SELECT * FROM customers
WHERE country = 'Mexico' OR country = 'UK' OR country = 'Canada';

-- b) Quais clientes são dos países 'Mexico', 'UK', 'Canada'
(utilize o método IN para resolver)
SELECT * FROM customers
WHERE country IN ('Mexico', 'UK', 'Canada');

-- ## WHERE + BETWEEN ## --
-- Essa combinação é uma alternativa ao uso do AND para filtrar
intervalos de números e datas.

-- 1. Selecione a tabela products
-- a) Quais produtos têm um unit_price entre 50 e 100 (utilize o
método AND para resolver)
SELECT * FROM products
WHERE unit_price ≥ 50 AND unit_price ≤ 100;

-- b) Quais produtos têm um unit_price entre 50 e 100 (utilize o
método BETWEEN para resolver)
SELECT * FROM products
WHERE unit_price BETWEEN 50 AND 100;

-- 2. Selecione a tabela orders
-- a) Quais pedidos foram feitos entre 01/01/1997 e 31/12/1997
(utilize o método AND para resolver)
SELECT * FROM orders
WHERE order_date ≥ '1997-01-01' AND order_date ≤ '1997-12-31';

-- b) Quais pedidos foram feitos entre 01/01/1997 e 31/12/1997
(utilize o método BETWEEN para resolver)
SELECT * FROM orders
WHERE order_date BETWEEN '1997-01-01' AND '1997-12-31';

/* MÓDULO 50 - POSTGRESQL
50. [PostgreSQL] Funções de Agregação e Agrupamentos no Postgres
1. COUNT
2. SUM
3. AVG, MIN e MAX
4. GROUP BY - Criando agrupamentos
5. GROUP BY, WHERE e HAVING - Utilizando filtros em agrupamentos
Feedback do módulo
*/
-- ## Funções de Agregação ## --
-- COUNT, SUM, AVG, MIN, MAX
```

```

-- 1. COUNT
-- a) Descubra a quantidade de clientes:
SELECT * FROM customers;

SELECT COUNT(*) FROM customers;

-- b) Descubra a quantidade de produtos:
SELECT COUNT(*) FROM products;

-- 2. SUM
-- a) Qual é a soma total de estoque de produtos?
SELECT * FROM products;

SELECT
    SUM(units_in_stock) AS "TotalEstoque"
FROM products;

-- b) Qual é a soma total de vendas de produtos? (lembre-se que
existem 2 tabelas de orders)
SELECT * FROM orders;

SELECT * FROM order_details;

SELECT
    SUM(quantity)
FROM order_details;

-- 3. AVG, MIN, MAX
-- a) Descubra a média, mínimo e máximo de unit_price da tabela
products
SELECT * FROM products;

SELECT
    AVG(unit_price) AS preco_medio,
    MIN(unit_price) AS menor_preco,
    MAX(unit_price) AS maior_preco
FROM products;

-- ## Criando agrupamento ## --
-- GROUP BY
-- 1. Faça um agrupamento para descobrir a quantidade total de
clientes por country.
SELECT * FROM customers;

SELECT DISTINCT country FROM customers;

SELECT
    country,
    COUNT(*)
FROM customers
GROUP BY country
ORDER BY COUNT(*) DESC;

```

-- 2. Faça um agrupamento para descobrir a quantidade total de clientes por title.

```
SELECT
    contact_title,
    COUNT(*)
FROM customers
GROUP BY contact_title
ORDER BY COUNT(*) DESC;
```

-- 3. Faça um agrupamento para descobrir a soma total de estoque por supplier_id.

```
SELECT * FROM products;
```

```
SELECT * FROM suppliers;
```

```
SELECT
    supplier_id,
    SUM(units_in_stock)
FROM products
GROUP BY supplier_id;
```

-- 4. Faça um agrupamento para descobrir a média de unit_price por supplier_id.

```
SELECT
    supplier_id,
    AVG(unit_price)
FROM products
GROUP BY supplier_id;
```

-- GROUP BY + WHERE vs GROUP BY + HAVING

-- 1. Faça um agrupamento para descobrir a quantidade total de clientes por country. O seu agrupamento deve considerar apenas os clientes de contact_title = 'Owner'.

```
SELECT * FROM customers;
```

```
SELECT
    country,
    COUNT(*)
FROM customers
WHERE contact_title = 'Owner'
GROUP BY country;
```

-- 2. Faça um agrupamento para descobrir a quantidade total de clientes por country. A query resultante deve conter apenas os países que têm mais de 10 clientes.

```
SELECT
    country,
    COUNT(*)
FROM customers
GROUP BY country
```



```
HAVING COUNT(*) ≥ 10;
```

```
/*
```

```
## Joins ## --
```

```
51. [PostgreSQL] Joins no Postgres
```

```
1. Introdução
```

```
2. LEFT JOIN x INNER JOIN x RIGHT JOIN x FULL JOIN
```

```
3. Exemplo prático 1
```

```
4. Exemplo prático 2
```

```
5. Exemplo prático 3
```

```
6. Join, Group By e Order By - Exemplo prático
```

```
7. Join, Group By, Where e Having - Exemplo prático
```

```
Feedback do módulo
```

```
*/
```

```
-- I. INTRODUÇÃO
```

```
-- Os JOINS no SQL têm como objetivo relacionar as diferentes  
tabelas dos nossos bancos de dados.
```

```
-- Com eles, conseguimos dar um passo além nas nossas análises,  
permitindo cruzar as informações das diferentes tabelas.
```

```
-- Para criar Joins, o primeiro passo é descobrir qual coluna as  
tabelas que queremos relacionar têm em comum.
```

```
-- Será através dessas colunas que o SQL saberá a forma como ele  
deve cruzar os dados.
```

```
-- Exemplo: as tabelas 'products' e 'order_details' possuem uma  
coluna em comum, chamada 'product_id'.
```

```
-- É daí que vêm os conceitos de CHAVE PRIMÁRIA e CHAVE ESTRANGEIRA,  
já explicado aqui no curso em módulos anteriores.
```

```
-- II. SINTAXE
```

```
-- A sintaxe mais simples para relacionar 2 tabelas (que tenham a  
'Coluna1' em comum) é a seguinte:
```

```
SELECT
```

```
    *
```

```
FROM
```

```
    Tabela_A
```

```
LEFT JOIN Tabela_B
```

```
ON Tabela_A.Coluna1 = Tabela_B.Coluna1;
```

```
-- Com a opção acima, estamos trazendo para uma mesma tabela TODAS  
as colunas das duas tabelas relacionadas, isso porque usamos o *.
```

```
-- Caso a gente queira escolher colunas específicas para visualizar  
na consulta final, seguimos a seguinte estrutura:
```

```
SELECT
```

```
    Tabela_A.Coluna1,
```

```
    Tabela_A.Coluna2,
```

```
    Tabela_A.Coluna3,
```

```

        Tabela_B.Coluna4
FROM
        Tabela_A
LEFT JOIN Tabela_B
ON Tabela_A.Coluna1 = Tabela_B.Coluna1;

```

-- Ou (Opção 2):

```

SELECT
        ta.Coluna1,
        ta.Coluna2,
        ta.Coluna3,
        tb.Coluna4
FROM
        Tabela_A ta
LEFT JOIN Tabela_B tb
ON ta.Coluna1 = tb.Coluna1;

```

-- LEFT JOIN x INNER JOIN x RIGHT JOIN x FULL JOIN

-- Imagine que temos uma tabela de VENDAS onde foram vendidos os produtos de id 1, 3 e 4

-- Imagine que temos uma tabela de PRODUTOS com produtos de id 1, 2 e 3

```

SELECT
        *
FROM vendas
XXXX JOIN produtos
ON vendas.id_produto = produtos.id_produto;

```

-- LEFT JOIN → Vai trazer as linhas da tabela de VENDAS (tabela da esquerda): vendas dos produtos de id 1, 3 e 4

-- INNER JOIN → Vai trazer as linhas de interseção entre a tabela de VENDAS e a de PRODUTOS: vendas dos produtos de id 1 e 3

-- RIGHT JOIN → Vai trazer as linhas da tabela de PRODUTOS (tabela da direita): vendas dos produtos de id 1, 2 e 3

-- FULL JOIN → Vai trazer todas as linhas das duas tabelas de VENDAS e PRODUTOS: vendas dos produtos de id 1, 2, 3 e 4

-- CRIANDO O PRIMEIRO JOIN

-- Faça um Join entre as tabelas 'products' e 'order_details'. Você deve retornar todas as colunas dessas duas tabelas.

```

SELECT * FROM products;

```

```

SELECT * FROM order_details;

```

```

SELECT
        *
FROM order_details
LEFT JOIN products
ON order_details.product_id = products.product_id;

```

-- 1. Avalie como as tabelas 'products' e 'categories' podem se

relacionar. Qual coluna as duas têm em comum?

```
SELECT * FROM products;  
SELECT * FROM categories;
```

-- Como poderíamos criar uma consulta que retornasse product_id, product_name, category_id, unit_price (tabela products) e category_name (categories)?

```
SELECT  
    products.product_id,  
    products.product_name,  
    products.category_id,  
    products.unit_price,  
    categories.category_name  
FROM products  
INNER JOIN categories  
ON products.category_id = categories.category_id;
```

-- Poderíamos dar nomes (aliases) para as tabelas:

```
SELECT  
    p.product_id,  
    p.product_name,  
    p.category_id,  
    p.unit_price,  
    c.category_name  
FROM products p  
LEFT JOIN categories c  
ON p.category_id = c.category_id;
```

-- 2. Avalie como as tabelas 'customers' e 'orders' podem se relacionar. Qual coluna as duas têm em comum?

-- Respostas: customer_id

```
SELECT * FROM customers;
```

```
SELECT * FROM orders;
```

-- Como poderíamos criar uma consulta que retornasse order_id, customer_id, order_date (tabela orders) e contact_name (customers)? Será que existem clientes que não fizeram nenhuma compra?

```
SELECT  
    orders.order_id,  
    orders.customer_id,  
    orders.order_date,  
    customers.contact_name  
FROM orders  
LEFT JOIN customers  
ON orders.customer_id = customers.customer_id;
```

```
SELECT DISTINCT customer_id FROM orders; -- 89 ids distintos  
(clientes que fizeram uma compra)
```

```
SELECT DISTINCT customer_id FROM customers; -- 91 ids distintos  
(clientes cadastrados na base)
```

-- Faça um Join entre as tabelas. Você seria capaz de descobrir qual o nome dos clientes que não fizeram nenhuma compra?

```
SELECT
    orders.order_id,
    customers.customer_id,
    orders.order_date,
    customers.contact_name
FROM orders
RIGHT JOIN customers
ON orders.customer_id = customers.customer_id;
```

-- ## Join + Group by + Order by ## --

-- Exemplo 1: Utilize os comandos JOIN, GROUP BY, ORDER BY para criar um agrupamento que retorne como resultado a quantidade total (SUM(quantity)) para cada product_name. Ordene o resultado, do maior para o menor, considerando a quantidade_total.

```
SELECT * FROM products;
```

```
SELECT * FROM order_details;
```

```
SELECT
    product_name,
    SUM(quantity) AS quantidade_total
FROM order_details
LEFT JOIN products
ON order_details.product_id = products.product_id
GROUP BY product_name
ORDER BY quantidade_total DESC;
```

-- ## Join + Group by + Where + Having ##

-- Exemplo 2: Ainda utilizando o código da análise anterior, faça o agrupamento de quantidade_total por product_name, considerando apenas os produtos da classe luxo (produtos com preço acima de R\$80)

```
SELECT
    p.product_name,
    SUM(od.quantity) AS quantidade_total
FROM order_details od
LEFT JOIN products p
ON od.product_id = p.product_id
WHERE p.unit_price ≥ 80
GROUP BY p.product_name
ORDER BY quantidade_total DESC;
```

-- Exemplo 3: Ainda utilizando o código da análise do Exemplo 1, faça o agrupamento de quantidade_total por product_name considerando apenas os produtos que tiveram uma quantidade_total maior ou igual a 1000. Obs.: estamos considerando a quantidade_total DEPOIS que o agrupamento foi feito.

```
SELECT
    product_name,
    SUM(quantity) AS quantidade_total
FROM order_details
```

```
LEFT JOIN products
    ON order_details.product_id = products.product_id
GROUP BY product_name
HAVING SUM(quantity) ≥ 1000
ORDER BY quantidade_total DESC;
```

```
/* MÓDULO POSTGRESQL
```

```
-- ## VIEWS ## --
```

```
52. [PostgreSQL] Views no Postgres
```

```
1. Create, Replace, Alter e Drop View
```

```
*/
```

```
-- I. INTRODUÇÃO
```

```
-- Views são formas de armazenar o resultado de uma query em uma espécie de tabela, dentro dos bancos de dados.
```

```
-- É uma forma muito prática de consultar e reaproveitar os resultados de consultas criadas de forma rotineira.
```

```
-- A view mostra sempre resultados atualizados da tabela.
```

```
-- II. SINTAXE:
```

```
-- Podemos criar (CREATE), atualizar (REPLACE), alterar (ALTER) e excluir (DROP) views.
```

```
-- Exemplo 1. Faça uma consulta à tabela 'products'. Você deve retornar apenas as colunas product_id, product_name e unit_price:
```

```
SELECT
```

```
    product_id,
    product_name,
    unit_price
```

```
FROM products;
```

```
-- Crie uma view a partir da consulta anterior
```

```
CREATE OR REPLACE VIEW vwprodutos AS
```

```
SELECT
```

```
    product_id,
    product_name,
    unit_price
```

```
FROM products;
```

```
-- Atualizar SCHEMAS (btn direito / REFRESH), e aparecerá em northwind / Schemas / Public / Views / vwprodutos.
```

```
SELECT * FROM vwprodutos;
```

```
-- Exemplo 2. Imagine que faltou incluir na sua view a coluna de units_in_stock. Altere a view criada anteriormente para fazer essa inclusão:
```

```
CREATE OR REPLACE VIEW vwprodutos AS
```

```
SELECT
```

```
    product_id,
    product_name,
    unit_price,
    units_in_stock
```

```
FROM products;
```

-- Atualizar SCHEMAS (btn direito / REFRESH), e aparecerá em northwind / Schemas / Public / Views / vwprodutos.

```
SELECT * FROM vwprodutos;
```

-- Exemplo 3. Altere o nome da sua view de 'vwprodutos' para 'vw_prod':

```
ALTER VIEW vwprodutos RENAME TO vw_prod;
```

-- Atualizar SCHEMAS (btn direito / REFRESH), e aparecerá em northwind / Schemas / Public / Views / vwprodutos.

```
SELECT * FROM vw_prod;
```

-- Exemplo 4. Exclua a view 'vw_prod':

```
DROP VIEW IF EXISTS vw_prod;
```

-- Atualizar SCHEMAS (btn direito / REFRESH), e aparecerá em northwind / Schemas / Public / Views / vwprodutos.

```
/* #≡≡≡ Operações CRUD ≡≡≡# --
```

```
53. [PostgreSQL] Operações CRUD no PostgreSQL
```

```
1. Introdução, Create e Drop Database
```

```
2. Tipos de Dados, Create e Drop Table
```

```
3. Constraints (Restrições)
```

```
4. Insert into, Update, Delete, Truncate e Drop (cascade)
```

```
Feedback do módulo
```

```
*/
```

-- Introdução

-- Operações CRUD são operações que conseguimos fazer em um banco de dados. Essa sigla significa o seguinte:

-- C reate: permite criar bancos de dados, tabelas ou exibições (Views);

-- R ead: permite ler os dados do banco de dados. Basicamente, foi o que mais fizemos no curso, através do SELECT;

-- U pdate: permite atualizar os dados do banco de dados, tabelas ou Views;

-- D elete: permite deletar dados de um banco de dados, tabelas ou Views.

-- No módulo anterior, vimos como criar Views (exibições) como os operadores CRUD.

-- 1. Criando um novo banco de dados:

-- Para criar um banco de dados, usamos o seguinte comando:

```
CREATE DATABASE teste; -- cria um novo banco de dados
```

-- Atualizar BD, clicar btn direito em DATABASES / REFRESH aparecerá o BD "teste" criado.

-- Para CONECTAR ao BD teste basta clicar com o btn esquerdo sobre BD teste.

-- Para Excluir / Deletar o BD teste tem que desconectar do BD.

```

-- Clicar brn direito em BD teste, Clicar em "Disconnect from
Database"
DROP DATABASE teste;          -- exclui um banco de dados criado
-- Para Excluir / Deletar o BD teste tem que desconectar do BD.
-- Clicar brn direito em BD teste, Clicar em "Disconnect from
Database"
-- Atualizar BD, clicar btn direito em DATABASES / REFRESH o BD
teste vai desaparecer.

-- ++++++
-- CRIAR NOVO BD(hashtag)

-- Vamos criar um banco de dados de exemplo para este módulo do
curso.
CREATE DATABASE hashtag;
-- Atualizar BD, clicar btn direito em DATABASES / REFRESH aparecera
o BD "hashtag" criado.
-- Para CONECTAR ao BD hashtag basta clicar com o btn esquerdo sobre
BD "hashtag".

-- 2. Tipos de dados para criação de tabelas:
-- Uma tabela tem como objetivo armazenar informações dispostas em
diferentes colunas.
-- Cada uma das colunas dessas tabelas será de um tipo específico.
-- SINTAXE:
CREATE TABLE tabela (coluna1 TIPO1, coluna2 TIPO2);

-- # Tipos de dados no PostgreSQL:
-- Quando criamos uma nova tabela, precisamos especificar quais são
as colunas que essa tabela deve conter.
-- Cada uma dessas colunas vai armazenar um tipo de dados
específico.
-- Os principais tipos de dados são listados abaixo:

-- INT:
-- Um número inteiro.

-- NUMERIC(M, D):
-- M é o número total de dígitos e D é a quantidade de casas
decimais permitidas.

-- Exemplo: 1500.59 → NUMERIC(6, 2)

-- VARCHAR(N): Uma string de comprimento VARIÁVEL (pode contar
letras, números e caracteres especiais).
-- O parâmetro N especifica o comprimento máximo da coluna em
caracteres.

-- DATE:
-- Uma data no formato: YYYY-MM-DD.

-- TIMESTAMP:

```

```
-- Uma combinação de data e hora. Formato: YYYY-MM-DD HH:MM:SS.

-- ++++++
-- CRIAR TABELAS NO BD hashtag
-- Clique com o btn esquerdo no BD hashtag para abrir.
-- Clique com o btn esquerdp em "Query Tool" para abrir uma
query no BD hashtag.
-- Agora essa query está dentro do BD hashtag.
-- Sabendo os principais tipos de dados, a criação das nossas
tabelas seguirá a estrutura abaixo:
```

```
CREATE TABLE alunos(
    ID_Aluno INT,
    Nome_Aluno VARCHAR(100),
    Email VARCHAR(100)
);
```

```
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH
```

```
CREATE TABLE cursos(
    ID_Curso INT,
    Nome_Curso VARCHAR(100),
    Preco_Curso NUMERIC(10, 2)
);
```

```
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH
```

```
CREATE TABLE matriculas(
    ID_Matricula INT,
    ID_Aluno INT,
    ID_Curso INT,
    Data_Cadastro DATE
);
```

```
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH
```

```
SELECT * FROM alunos;
SELECT * FROM cursos;
SELECT * FROM matriculas;
```

```
-- MAS... ainda podemos incluir CONSTRAINTS, ou seja, restrições na
criação das tabelas.
```

```
DROP TABLE alunos;
DROP TABLE cursos;
DROP TABLE matriculas;
```

```
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH
```

```
-- # 3. Restrições (Constraints) são regras aplicadas nas colunas de
uma tabela
```

```
-- Ex. 1: Podemos especificar que uma coluna não pode ter
valores NULL;
```



```

-- Ex. 2: Podemos especificar que uma coluna deverá ser uma
chave primária ou chave estrangeira
-- São usadas para limitar os tipos de dados que são inseridos.

-- # Principais:

-- # I. NOT NULL:
-- A Constraint NOT NULL faz com que uma coluna não aceite valores
NULL. Ela identifica que nenhum valor foi definido.
-- A Constraint NOT NULL obriga um campo a sempre possuir um valor.
-- Dessa forma, uma coluna com restrição NOT NULL não aceita valores
vazios.
-- Nome_Cliente VARCHAR(100) NOT NULL

-- # II. PRIMARY KEY (Chave Primária):
-- A PRIMARY KEY identifica de forma única cada registro em uma
tabela do banco de dados.
-- Chaves primárias devem conter valores únicos.
-- Uma coluna de chave primária não pode conter valores NULL.
-- Cada tabela deve conter 1 e apenas 1 chave primária.

-- # III. FOREIGN KEY (Chave Estrangeira):
-- Uma FOREIGN KEY em uma tabela é um campo que aponta para uma
chave primária em outra tabela.

-- # CREATE TABLE:
-- # 4. Criando as tabelas no banco de dados:

CREATE TABLE alunos(
    ID_Aluno INT,
    Nome_Aluno VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    PRIMARY KEY(ID_Aluno)
);
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH

CREATE TABLE cursos(
    ID_Curso INT,
    Nome_Curso VARCHAR(100) NOT NULL,
    Preco_Curso NUMERIC(10, 2) NOT NULL,
    PRIMARY KEY(ID_Curso)
);
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH.

CREATE TABLE matriculas(
    ID_Matricula INT,
    ID_Aluno INT NOT NULL,
    ID_Curso INT NOT NULL,
    Data_Cadastro DATE NOT NULL,
    PRIMARY KEY(ID_Matricula),

```

```

        FOREIGN KEY(ID_Aluno) REFERENCES alunos(ID_Aluno),
        FOREIGN KEY(ID_Curso) REFERENCES cursos(ID_Curso)
    );
-- Atualizar BD hashtag, clicar com btn direito em BD hashtag /
REFRESH

-- # INSERT INTO

-- 5. Inserindo dados nas tabelas.
INSERT INTO alunos(ID_Aluno, Nome_Aluno, Email)
VALUES
    (1, 'Ana' , 'ana123@gmail.com' ),
    (2, 'Bruno' , 'bruno_vargas@outlook.com' ),
    (3, 'Carla' , 'carlinha1999@gmail.com' ),
    (4, 'Diego' , 'diicatroneves@gmail.com' ),
    (5, 'Charmilton' , 'Charmilton@gmail.com' ),
    (6, 'Maria Fernandes' , 'maria19@gmail.com' ),
    (7, 'Analia' , 'analia@gmail.com' ),
    (8, 'Antonio Oliveira' , 'antonio@gmail.com' );

SELECT * FROM alunos;

INSERT INTO cursos(ID_Curso, Nome_Curso, Preco_Curso)
VALUES
    (1, 'Excel' , 100),
    (2, 'VBA' , 200),
    (3, 'PowerBI' , 150),
    (4, 'Full Stak' , 450),
    (5, 'Python' , 1150);

SELECT * FROM cursos;

INSERT INTO matriculas(ID_Matricula, ID_Aluno, ID_Curso,
Data_Cadastro)
VALUES
    (1, 1, 1, '2021/03/11' ),
    (2, 1, 2, '2021/06/21' ),
    (3, 2, 3, '2021/01/08' ),
    (4, 3, 1, '2021/04/03' ),
    (5, 4, 1, '2021/05/10' ),
    (6, 4, 3, '2021/05/10' ),
    (7, 5, 1, '2021/03/11' ),
    (8, 5, 2, '2021/06/21' ),
    (9, 6, 3, '2021/01/08' ),
    (10, 7, 1, '2021/04/03' ),
    (11, 8, 1, '2021/05/10' ),
    (12, 8, 3, '2021/05/10' );

SELECT * FROM matriculas;

-- # UPDATE:
-- 6. Atualizando dados de uma tabela com o UPDATE:

```

```
SELECT * FROM alunos;
SELECT * FROM cursos;
SELECT * FROM matriculas;
```

```
UPDATE cursos
SET Preco_Curso = 300
WHERE ID_Curso = 1;
```

```
SELECT * FROM cursos;
```

```
-- # DELETE:
-- 7. Deletando registros de uma tabela:
```

```
SELECT * FROM matriculas;
SELECT * FROM alunos;
SELECT * FROM cursos;
```

```
DELETE FROM matriculas
WHERE ID_Matricula = 6;
```

```
SELECT * FROM matriculas;
```

```
-- # TRUNCATE
```

```
-- 8. Deletando todos os registros da tabela de uma vez, mas a
tabela continua existindo:
```

```
SELECT * FROM matriculas;
```

```
TRUNCATE TABLE matriculas; -- Não apaga a tabela só os dados
```

```
SELECT * FROM matriculas;
```

```
-- # DROP
```

```
-- 9. Deletando tabelas do banco de dados
```

```
-- Atenção: Cuidado ao excluir uma tabela que tenha restrições de
PRIMARY KEY!!!
```

```
DROP TABLE alunos CASCADE;
DROP TABLE cursos CASCADE;
DROP TABLE matriculas;
```

```
/*
```

54. [PostgreSQL] Funções de Número, Texto e Data

1. Funções de Número – Ceiling, Floor, Round, Trunc

2. Funções de Texto – Upper, Lower, Length, Initcap, Replace, Substring e Strpos

3. Funções de Data – Current_Date, Age, Date_Part

Feedback do módulo

```
*/
```

```
/*
```

Em PostgreSQL, as funções CEILING, FLOOR, ROUND e TRUNC são usadas para manipular números e arredondá-los. CEILING retorna o menor inteiro maior ou igual ao número dado. FLOOR retorna o maior inteiro menor ou igual ao número dado. ROUND arredonda para o inteiro mais

próximo ou para um número específico de casas decimais. TRUNC remove a parte decimal do número, truncando-o para um número inteiro ou para um número específico de casas decimais.

Funções de Arredondamento:

CEILING(number):

* Arredonda o número para cima, para o inteiro mais próximo maior ou igual ao número original. Por exemplo, CEILING(4.2) retornará 5.

FLOOR(number):

* Arredonda o número para baixo, para o inteiro mais próximo menor ou igual ao número original. Por exemplo, FLOOR(4.7) retornará 4.

ROUND(number, precision):

* Arredonda o número para o inteiro mais próximo ou para o número especificado de casas decimais. O parâmetro precision (opcional) indica quantas casas decimais manter após o arredondamento. Se precision for omitido, arredonda para o inteiro mais próximo. Por exemplo, ROUND(4.678, 2) retornará 4.68, enquanto ROUND(4.2) retornará 4.

TRUNC(number, precision):

* Trunca o número para o número especificado de casas decimais, removendo a parte decimal. O parâmetro precision (opcional) indica quantas casas decimais manter após o truncamento. Se precision for omitido, remove a parte decimal. Por exemplo, TRUNC(4.678, 2) retornará 4.67, enquanto TRUNC(4.678) retornará 4.

Exemplos:

*/

```
SELECT CEILING(4.2); -- Retorna 5
SELECT FLOOR(4.7);  -- Retorna 4
SELECT ROUND(4.678, 2); -- Retorna 4.68
SELECT ROUND(4.2);  -- Retorna 5
SELECT TRUNC(4.678, 2); -- Retorna 4.67
SELECT TRUNC(4.678); -- Retorna 4
```

/*

Observações:

* As funções CEILING e FLOOR sempre retornam um inteiro, independentemente do número de casas decimais do argumento.

* A função ROUND arredonda para cima ou para baixo, com base no valor da casa decimal mais significativa ignorada.

* A função TRUNC remove a parte decimal sem arredondar, sempre truncando o número para o número especificado de casas decimais ou para um inteiro.

* A função TRUNC pode ser usada com um parâmetro de precisão negativo, onde nesse caso, ela define para 0 os dígitos à direita da vírgula decimal e substitui os dígitos à esquerda da vírgula decimal, com base no número de casas decimais especificado (positivo), Neon Postgres.

*/

```
-- INÍCIO MÓDULO
-- Funções de Número, Texto e Data
-- Aula 1 - Funções de Número
```

```
-- Ceiling, Floor, Round, Trunc
select * from products;
```

```
select
    avg(unit_price),
    ceiling(avg(unit_price)),
    floor(avg(unit_price)),
    round(cast(avg(unit_price) as numeric), 3),
    trunc(cast(avg(unit_price) as numeric), 3)
from products;
```

```
-- Aula 2 - Funções de Texto
-- Upper, Lower, Length, Initcap
/*
```

Em PostgreSQL, as funções UPPER, LOWER, LENGTH e INITCAP são usadas para manipulação de strings. UPPER converte uma string para maiúsculas, LOWER para minúsculas, LENGTH retorna o comprimento da string e INITCAP converte a primeira letra de cada palavra para maiúscula e o restante para minúscula.

Exemplos:

UPPER(string): Converte a string para maiúsculas.

```
*/
SELECT UPPER('hello world');
-- Resultado: HELLO WORLD
```

```
/*
LOWER(string): Converte a string para minúsculas.
*/
```

```
SELECT LOWER('HELLO WORLD');
-- Resultado: hello world
```

```
/*
LENGTH(string): Retorna o número de caracteres na string.
*/
```

```
SELECT LENGTH('hello');
-- Resultado: 5
```

```
/*
INITCAP(string): Converte a primeira letra de cada palavra para maiúscula e o restante para minúscula.
*/
```

```
SELECT INITCAP('hello world');
-- Resultado: Hello World
```

```
/*
Observações:
```

- * As funções UPPER e LOWER são sensíveis à localidade, ou seja, o comportamento pode variar dependendo da configuração de localidade do banco de dados.

- * A função INITCAP considera palavras como sequências de caracteres delimitadas por espaços ou outros caracteres não alfanuméricos.

- * Essas funções são frequentemente usadas para formatação de texto, comparação de strings (especialmente quando ignorando maiúsculas/minúsculas) e preparação de dados para exibição.

```
*/
select * from employees;
```

```
select
    first_name,
    upper(first_name),
    lower(first_name),
    length(first_name),
    initcap('sql impressionador')
from employees;
```

-- Replace

/*

Em PostgreSQL, a função REPLACE é usada para substituir todas as ocorrências de uma substring em uma string por outra substring. A sintaxe básica é REPLACE(string, substring_antiga, nova_substring). Essa função é útil para limpeza de dados, formatação de texto e outras tarefas de manipulação de strings.

Sintaxe:

*/

```
REPLACE(string, substring_antiga, nova_substring)
```

/*

Parâmetros:

string: A string onde a substituição será realizada.

substring_antiga: A substring que será substituída.

nova_substring: A substring que substituirá a antiga.

Exemplos:

Substituindo texto:

*/

```
SELECT REPLACE('Olá, mundo!', 'mundo', 'PostgreSQL');
-- Resultado: 'Olá, PostgreSQL!'
```

/*

Substituindo múltiplos caracteres:

*/

```
SELECT REPLACE('banana', 'na', 'ta');
-- Resultado: 'batata'
```

/*

Substituindo em uma tabela:

*/

```
UPDATE sua_tabela
SET coluna_texto = REPLACE(coluna_texto, 'texto_antigo',
'texto_novo')
WHERE condição;
```

/*

A função REPLACE é case-sensitive, ou seja, diferencia maiúsculas de minúsculas. Se você precisar de uma substituição que não diferencia maiúsculas de minúsculas, pode ser necessário usar outras abordagens, como combinar funções de conversão de caixa com a função REPLACE ou utilizar expressões regulares.

*/

```
select * from customers;
```

```
select
    contact_name,
```

```

        contact_title,
        replace(contact_title, 'Owner', 'CEO') -- Substitui 'Owner' por
'CEO'
from customers;

```

```

select
    contact_name,
    contact_title,
    replace(contact_title, 'Owner', 'CEO') AS
Substitui_Owner_Por_CEO -- Substitui 'Owner' por 'CEO'
from customers;

```

-- Substring e Strpos

/*

Em PostgreSQL, a função substring extrai uma parte de uma string, enquanto strpos (ou position) retorna a posição de uma substring dentro de outra.

Função SUBSTRING()

A função substring() permite extrair uma parte de uma string, especificando a posição inicial e o comprimento da substring desejada.

*/

```

    substring(string, start_position, length)

```

/*

string: A string da qual você deseja extrair a substring.

start_position: A posição inicial da substring (o primeiro caractere é a posição 1).

length: O número de caracteres a serem extraídos. Se omitido, a função extrai até o final da string a partir da posição inicial.

Exemplos:

*/

```

SELECT substring('Olá, mundo!', 1, 5); -- Resultado: 'Olá, '
SELECT substring('Olá, mundo!', 7);    -- Resultado: 'mundo!'
SELECT substring('PostgreSQL', 1, 4);  -- Resultado: 'Post'

```

/*

Função STRPOS() (ou POSITION())

* A função strpos() (ou position()) retorna a posição inicial de uma substring dentro de uma string.

*/

```

    strpos(string, substring)

```

```

    -- ou

```

```

    position(substring in string)

```

/*

string: A string na qual você deseja procurar.

substring: A substring que você está procurando.

Exemplos:

*/

```

SELECT strpos('Olá, mundo!', 'mundo');
-- Resultado: 7
SELECT position('mundo' in 'Olá, mundo!');
-- Resultado: 7
SELECT strpos('PostgreSQL', 'Post');

```

```
-- Resultado: 1
SELECT strpos('PostgreSQL', 'Java');
-- Resultado: 0 (não encontrado)
```

```
/*
```

Diferenças e similaridades entre as funções

substring() extrai uma parte da string, enquanto strpos() (ou position()) localiza a posição de uma substring.

substring() requer um início e um comprimento (ou apenas um início) para extrair a substring. strpos() (ou position()) requer apenas a substring a ser procurada e a string onde procurar.

strpos() (ou position()) retorna 0 se a substring não for encontrada.

substring() pode retornar uma string vazia ou NULL se a posição inicial for inválida ou o comprimento exceder o tamanho da string.

Observação: Em alguns casos, você pode precisar usar a função regexp_substr para extrair substrings com base em expressões regulares.

```
*/
```

```
select
  'ABC-9999',
  left('ABC-9999', 3),
  right('ABC-9999', 4);
```

```
select
  'ABC-9999',
  substring('ABC-9999', 1, strpos('ABC-9999', '-') -1),
  substring('ABC-9999', strpos('ABC-9999', '-') +1, 100),
  strpos('ABC-9999', '-');
```

```
-- Aula 3 - Funções de Data
```

```
-- Current_Date, Age, Date_Part
```

```
-- ++++++
```

```
/*
```

Em PostgreSQL, as funções CURRENT_DATE, AGE e DATE_PART são usadas para manipular e extrair informações de datas e horários.

CURRENT_DATE retorna a data atual, AGE calcula a diferença entre duas datas como um intervalo, e DATE_PART extrai uma parte específica de uma data ou horário.

1. CURRENT_DATE:

- * Retorna a data atual no fuso horário da sessão do banco de dados.

- * É uma função simples que não requer argumentos e retorna um valor do tipo DATE.

Exemplo: SELECT CURRENT_DATE;

2. AGE:

- * Calcula a diferença entre duas datas ou horários e retorna um intervalo.

- * Sintaxe: AGE(data_fim, data_inicio) ou AGE(data) (onde a data atual é usada como data_fim).

Exemplo: SELECT AGE('2024-07-31', '2000-01-01'); (retorna um intervalo de 24 anos, 6 meses e 30 dias).

3. DATE_PART:

- * Extrai uma parte específica de um valor de data ou horário.
- * Sintaxe: DATE_PART('campo', data_ou_horario).
campo especifica qual parte extrair (ex: 'year', 'month', 'day', 'hour', 'minute', etc.).
- * Exemplo: SELECT DATE_PART('year', '2024-07-31 10:30:00');
(retorna 2024).
- * Outros valores permitidos para campo: 'century', 'decade', 'millisecond', 'microsecond', 'dow', 'doy', 'epoch', 'isodow', 'isoyear', 'timezone', 'timezone_hour', 'timezone_minute'.

Exemplo prático usando as três funções:

```

SELECT
    CURRENT_DATE AS data_nasc,
    AGE(birth_date) AS idade,
    DATE_PART('year', birth_date) AS ano,
    DATE_PART('month', birth_date) AS mes,
    DATE_PART('day', birth_date) AS dia
FROM employees;

```

/*
Este exemplo mostra como usar as três funções em uma única consulta para obter a data atual, a idade calculada a partir de duas datas, e o ano, mês e dia de uma data específica.

```

*/
select * from employees;

```

```

select
    first_name,
    birth_date,
    current_date,
    age(birth_date),
    date_part('day', birth_date),
    date_part('month', birth_date),
    date_part('year', birth_date)
from employees;

```

```

-- ++++++
/*

```

55. [PostgreSQL] Subqueries

1. O que é uma Subquery
2. Subquery: Cláusula WHERE
3. Subquery: Cláusula FROM
4. Subquery: Cláusula SELECT
5. Subquery: Corrigindo a análise de pedidos acima da média

Feedback do módulo

```

*/
/*

```

Em PostgreSQL, uma subquery, também conhecida como subseleção ou consulta aninhada, é uma consulta SQL que está inserida dentro de outra consulta, seja ela uma instrução SELECT, INSERT, UPDATE ou DELETE. Basicamente, a subquery é executada primeiro, e o resultado que ela retorna é utilizado pela consulta principal.

Em termos mais simples: Imagine uma receita de bolo. A subquery é

como misturar os ingredientes antes de colocar a massa no forno (a consulta principal). Você precisa ter os ingredientes certos (o resultado da subquery) para poder fazer o bolo (a consulta principal).

Características:

Aninhamento:

- * Subqueries são sempre inseridas dentro de outras consultas, criando um nível de aninhamento.

Execução:

- * A subquery é executada primeiro e seu resultado é usado pela consulta principal.

Utilização:

- * Podem ser usadas em diversas partes de uma consulta, como na cláusula WHERE, FROM, SELECT, entre outras.

Finalidade:

- * Permitem realizar operações mais complexas e sofisticadas, que seriam difíceis ou impossíveis de serem feitas com uma única consulta.

Exemplos de uso:

- * Filtragem de dados: Usar o resultado de uma subquery para filtrar os dados da consulta principal.

Cálculos complexos: Realizar cálculos complexos utilizando o resultado de uma subquery.

- * Inserção, atualização e exclusão: Usar o resultado de uma subquery para inserir, atualizar ou excluir dados em uma tabela.

Exemplo prático:

- * Suponha que você queira encontrar os produtos que têm o maior preço em uma tabela de produtos. Você poderia usar uma subquery para encontrar o preço máximo e, em seguida, usar essa informação na consulta principal para selecionar os produtos com esse preço.

Vantagens:

Flexibilidade:

- * Permite criar consultas mais complexas e adaptáveis a diferentes cenários.

Reutilização:

- * O resultado de uma subquery pode ser reutilizado em várias partes da consulta principal.

Legibilidade:

- * Em alguns casos, pode tornar a consulta mais legível, especialmente quando a lógica é complexa.

Desvantagens:

Desempenho:

- * Em alguns casos, subqueries podem ter um impacto negativo no desempenho da consulta, especialmente se não forem otimizadas.

Complexidade:

- * Consultas com muitas subqueries podem se tornar difíceis de entender e manter.

Em resumo, subqueries são uma ferramenta poderosa no PostgreSQL para

lidar com consultas complexas e flexíveis. No entanto, é importante usá-las com cuidado e otimizá-las para garantir o melhor desempenho. */

-- Aula 1 - O que é uma Subquery

-- Subqueries no SQL são queries dentro de queries. É a possibilidade de reaproveitar o resultado de uma query (select) dentro de outra.

-- Exemplo: Quais produtos têm um preço acima da média?

```
select * from products;
```

```
select avg(unit_price) from products; -- 28.833896 ...
```

```
select * from products
```

```
where unit_price ≥ (select avg(unit_price) from products);
```

-- Aula 2 - Subquery: Cláusula WHERE

-- Exemplo: Quais pedidos têm uma quantidade vendida acima da quantidade vendida média?

```
select * from order_details;
```

```
select avg(quantity) from order_details; -- 23.81299 ...
```

```
select * from order_details
```

```
where quantity ≥ (
    select
        avg(quantity)
    from order_details
);
```

-- Aula 3 - Subquery: Cláusula FROM

-- Exemplo: Qual é a média de clientes de acordo com o cargo?

```
select * from customers;
```

```
select
```

```
    avg(total_clientes)
from (
    select
        contact_title,
        count(*) total_clientes
    from customers
    group by contact_title) t;
```

-- Aula 4 - Subquery: Cláusula SELECT

-- Exemplo: Faça uma consulta à tabela products e adicione uma coluna que contenha a média geral de preço dos produtos.

```
select * from products;
```

```
select
```

```
    *,
    (select avg(unit_price) from products) media_preco
from products;
```

-- Aula 5 - Subquery: Corrigindo a análise de pedidos acima da média
-- Exemplo: Quais pedidos têm uma quantidade vendida acima da
quantidade vendida média?

```
select * from order_details;
```

-- Quantidade no Pedido

```
select
```

```
    order_id,  
    sum(quantity) total_vendido  
from order_details  
group by order_id;
```

--

```
select
```

```
    avg(total_vendido)  
from (  
    select  
        order_id,  
        sum(quantity) total_vendido  
    from order_details  
    group by order_id) t; -- Media de quantidade =
```

```
61.827710843373....
```

--

```
select
```

```
    order_id,  
    sum(quantity)  
from order_details  
group by order_id  
having sum(quantity) ≥ (  
    select  
        avg(total_vendido)  
    from (  
        select  
            order_id,  
            sum(quantity) total_vendido  
        from order_details  
        group by order_id) t);
```

-- ++++++

```
/*
```

56. [PostgreSQL] Variáveis e Blocos Anônimos

1. Variáveis, Datatypes e Blocos Anônimos

2. Blocos Anônimos - Exemplos

Feedback do módulo

```
*/
```

```
/*
```

Em PostgreSQL, variáveis são espaços de memória nomeados utilizados para armazenar dados dentro de um bloco de código, como funções ou blocos anônimos. Tipos de dados (datatypes) definem o tipo de valor que uma variável pode conter (inteiro, texto, etc.). Blocos anônimos são trechos de código que podem ser executados sem a necessidade de serem definidos como uma função ou procedimento, permitindo a execução de código procedural dentro do PostgreSQL.

Variáveis:

- * São usadas para armazenar valores temporários durante a execução de um bloco de código PL/pgSQL.
- * Devem ser declaradas antes de serem usadas, especificando seu nome e tipo de dado.
- * Podem ter um valor inicial definido ou receber um valor posteriormente.
- * O escopo de uma variável é limitado ao bloco em que ela é declarada.

Tipos de Dados:

- * Definem o tipo de dado que uma variável pode armazenar, como números inteiros (INTEGER), textos (VARCHAR), datas (DATE), etc.
- * Permitem que o PostgreSQL valide os valores atribuídos às variáveis, garantindo a integridade dos dados.
- * Exemplos comuns incluem INTEGER, VARCHAR, BOOLEAN, DATE, TIMESTAMP, e muitos outros.

Blocos Anônimos:

- * São trechos de código PL/pgSQL que podem ser executados diretamente, sem a necessidade de estarem associados a uma função ou procedimento.
- * Geralmente usados para realizar tarefas pontuais ou testes.
- * Possuem uma seção de declaração (opcional) onde as variáveis são declaradas, e uma seção de corpo onde o código é executado.

Sintaxe básica:

```
*/  
    <<nome_do_bloco>>  
    DECLARE  
        -- Declaração de variáveis  
    BEGIN  
        -- Código a ser executado  
    END;
```

/*

O ponto e vírgula (;) é obrigatório ao final de cada instrução dentro do bloco

*/

-- INÍCIO MÓDULO

-- Variáveis e Blocos Anônimos

-- Aula 1 - Variáveis, Datatypes e Blocos Anônimos

-- Variáveis são pedaços de memória onde armazenamos alguma informação. Uma variável está sempre associada a um tipo de dado em particular.

-- Os tipos mais comuns de dados são:

-- 1. Numéricos: int e decimal

-- 2. Textos: varchar(n)

-- 3. Datas: date

-- Bloco Anônimo (Introdução)

-- No PostgreSQL, é possível criar os chamados Blocos Anônimos, blocos de códigos que são a base para Functions e Procedures.
-- Abaixo, temos a estrutura de um bloco anônimo.

```
<<label>>
declare
    declaracao;
begin
    corpo do código;

end label;
```

-- Cada bloco possui duas sessões: declaração e corpo.
-- A sessão de declaração é opcional e é onde declaramos todas as variáveis usadas no corpo do código.
-- A sessão do corpo é obrigatória e é onde criamos os nossos códigos.
-- Em ambas as sessões é obrigatório o uso do ponto e vírgula ao final de cada instrução.

-- Abaixo, temos um exemplo de um bloco anônimo:

```
do $$
declare
    nome varchar(100);
    salario decimal;
    data_contratacao date;
begin
    nome := 'André';
    salario := 3500;
    data_contratacao := '25-10-2018';
    raise notice 'O funcionário % foi contratado em % e recebe um
salário de R$ %.', nome, data_contratacao, salario;
end $$;
```

-- Aula 2 - Blocos Anônimos - Exemplos

-- Exemplo 1: Criando uma calculadora simples de valor vendido. Utilize as variáveis 'quantidade', 'preco', 'valor_vendido' e 'vendedor' para isso.

```
do $$
declare
    quantidade int := 50;
    preco decimal := 100;
    valor_vendido int;
    vendedor varchar(100) := 'Bruna';

begin
    valor_vendido := quantidade * preco;

    raise notice 'O vendedor % vendeu o total de R$ %.', vendedor,
```

```

valor_vendido;

end $$;

-- Exemplo 2: Quantos produtos têm o preço acima da média de preços?
do $$
declare
    media_preco decimal;
    qtd_produtos_acima_media int;

begin
    media_preco = (select avg(unit_price) from products);
    qtd_produtos_acima_media = (select count(*) from products where
unit_price ≥ media_preco);

    raise notice 'A quantidade de produtos com preço acima da média
é de: % produtos.', qtd_produtos_acima_media;

end $$;

-- ++++++
/*
57. [PostgreSQL] Functions
1. O que é e como criar uma Function
2. Formas de chamar uma Function e como excluir uma Function
3. Functions que retornam uma tabela e como copiar tipo de dados
para uma variável
Feedback do módulo
*/
-- INÍCIO MÓDULO
-- User-Defined Functions

-- Aula 1 - Criando uma Function
-- Além das funções de números, textos, datas, agregação, presentes
no SQL, também é possível criar funções personalizadas que realizam
algum tipo de cálculo. A essas funções damos o nome de User-Defined
Functions, ou, Funções Definidas pelo Usuário.

-- A sintaxe para criar uma função é a seguinte:
create or replace function nome_funcao(parametros)
    returns tipo_dado
    language plpgsql
as
$$
declare
    declaracao de variaveis
begin
    codigos
    return
end $$;

-- Exemplo: Crie uma Function que analisa o estoque de produtos.

```

Essa Function deve retornar o total de produtos que possuem um total de estoque entre um estoque mínimo e um estoque máximo, ambos definidos pelo usuário da Function.

```
select * from products;

create or replace function analise_estoque(estoque_min int,
estoque_max int)
returns int
language plpgsql
as
$$
declare
    contagem_estoque int;
begin
    contagem_estoque = (select count(*) from products where
units_in_stock between estoque_min and estoque_max);
    return contagem_estoque;
end $$;
-- Atualizar Functios REFRESH
-- Executar função criada.
select analise_estoque(10, 50); -- resultado 42

select count(*) from products; -- resultado 77

select count(*) - analise_estoque(10, 50) from products; --
resultado 77 - 42 = 35

-- Aula 2 - Formas de chamar uma Function e como excluir uma
Function
-- Existem 3 formas de chamar uma função:
-- i) Usando a notação por posição:
select analise_estoque(20, 50);

-- ii) Usando a notação por nome do parâmetro:
select analise_estoque(estoque_min := 20, estoque_max := 50);

-- iii) Usando a notação mista:
select analise_estoque(20, estoque_max := 50);
-- A notação mistra não é recomendável, pois se fizer o contrário,
dá erro:
-- select analise_estoque(estoque_min := 20, 50);

-- Excluindo uma Function:
drop function analise_estoque;
drop function if exists analise_estoque;

-- Aula 3 - Functions que retornam uma tabela
-- É possível criar funções que retornam tabelas.
-- A sintaxe para criar uma função deste tipo é a seguinte:
create or replace function nome_funcao(parametros)
returns table (lista_de_colunas)
language plpgsql
```



```

as
$$
declare
    declaracao de variaveis
begin
    codigos
    return query
end $$;
-- Atualizar Functios REFRESH

-- Exemplo: Crie uma Function que retorne uma tabela contendo a
-- lista de clientes que têm o contact_title = 'Owner'.
-- A tabela retornada pela Fucntion deve conter apenas as colunas de
-- id, nome, telefone e cargo dos clientes.
select * from customers;

create or replace function busca_clientes (title varchar)
returns table
(
    id customers.customer_id%type,
    nome customers.contact_name%type,
    telefone customers.phone%type,
    cargo customers.contact_title%type
)
language plpgsql
as $$
begin
    return query
        select
            customer_id,
            contact_name,
            phone,
            contact_title
        from customers
        where contact_title = title;
end $$;
-- Atualizar Functios REFRESH
-- Executar função criada.
select * from busca_clientes('Owner');

-- Excluindo uma Function:
drop function busca_clientes;

```

/*

58. [PostgreSQL] Transactions e Procedures

1. O que é uma Transação?
2. O que é, como criar e excluir uma Procedure
3. Exemplo – Procedure para registro de transferência bancária

Feedback do módulo

*/

-- INÍCIO MÓDULO

-- Transactions e Procedures

-- Aula 1 - O que é uma Transação?

-- Uma transação em um banco de dados é uma unidade de trabalho que consiste em uma ou mais operações.

-- Um exemplo clássico de uma transação é uma transferência bancária de uma conta para outra. Se o remetente transfere um valor de X reais a um destinatário, este destinatário deverá receber exatamente esta quantia de X reais, nem mais, nem menos.

```
create table contas(  
    id int,  
    nome varchar(100),  
    saldo decimal);
```

```
select * from contas;
```

```
insert into contas(id, nome, saldo)  
values(1, 'Ana', 5000);
```

```
begin transaction;  
insert into contas(id, nome, saldo)  
values(2, 'Bruno', 10000);
```

```
commit;
```

```
rollback;
```

-- Para fazer com que as mudanças fiquem visíveis em outras sessões, é necessário utilizar o comando COMMIT.

-- Já para desfazer uma transação, usamos o comando ROLLBACK.

-- Aula 2 - O que é e como criar uma Procedure

-- Até aqui aprendemos como criar blocos anônimos e functions. Porém, functions e blocos anônimos não são capazes de executar transações.

-- Ou seja, dentro de uma function, não podemos iniciar uma transação, ou dar commit/rollback nela.

-- A sintaxe para criação de uma Procedure é a seguinte:

```
create or replace procedure nome_procedure(parametros)  
language plpgsql  
as $$  
declare  
    declaracao de variaveis  
begin  
    corpo de código  
end $$;  
/*  
Para excluir, a sintaxe é:  
*/  
drop procedure nome_procedure;
```

-- Exemplo: Crie uma procedure que cadastre um novo cliente na tabela Contas.

```
create or replace procedure cadastra_cliente(novo_id int,  
novo_cliente varchar(100), saldo_inicial decimal)  
language plpgsql
```

```
as $$
```

```
begin
```

```
    insert into contas(id, nome, saldo) values  
        (novo_id, novo_cliente, saldo_inicial);
```

```
    commit;
```

```
end $$;
```

```
call cadastra_cliente(3, 'Caio', 300);
```

```
select * from contas;
```

```
drop procedure cadastra_cliente;
```

-- Exemplo: Crie uma Procedure que controle transferências bancárias entre duas contas.

```
select * from contas;
```

```
create or replace procedure transferencia(remetente int,  
destinatario int, valor decimal)
```

```
language plpgsql
```

```
as $$
```

```
begin
```

```
    -- subtrair o montante transferido pelo remetente
```

```
    update contas
```

```
    set saldo = saldo - valor
```

```
    where id = remetente;
```

```
    -- adicionar o montante transferido para o destinatário
```

```
    update contas
```

```
    set saldo = saldo + valor
```

```
    where id = destinatario;
```

```
    commit;
```

```
end $$;
```

```
call transferencia(1, 2, 500);
```

```
select * from contas;
```

```
drop procedure transferencia;
```

```
Drop table contas;
```